

Enhanced Edge IOT system for hands free control and energy monitoring

ATIKSH SINGH

Computer Science and Engineering
School of Computer Science
Phagwada, Punjab, India
atiksh.singhh19@gmail.com

ARJUN SINGH

Computer Science and Engineering
School of Computer Science
Phagwada, Punjab, India
singharjun7002@gmail.com

MANISH NEHRA

Computer Science and Engineering
School of Computer Science
Phagwada, Punjab, India
nehramanish50@gmail.com

RISHABH JAIN

Computer Science and Engineering
School of Computer Science
Phagwada, Punjab, India
rishabhjain08040@gmail.com

Abstract

This project introduces an intelligent home automation and safety system that enables contactless appliance control using hand gestures while continuously monitoring electrical energy usage. The system measures current, power, and total energy consumed using the ACS712 current sensor and displays the values on both an LCD display and the Blynk IoT application via Wi-Fi. To enhance safety, an MQ135 gas sensor is integrated to detect LPG or smoke. When gas concentration exceeds a defined threshold, the system triggers a buzzer alert and automatically switches on an exhaust fan to reduce gas accumulation, while the Blynk app updates the gas status in real time. The app also visualizes detailed graphs such as power vs time, energy vs time, current vs time, and gas level vs time, offering deeper analytical insight to the user. By combining gesture-based control, IoT-enabled monitoring, and automated safety response, this project contributes to a modern smart-home framework prioritizing convenience, efficiency, and user protection.

Keywords: Gesture Control, IoT, Energy Monitoring, Smart Home, MQ135, Home Automation

I. Introduction

A. Background of Study and Motivation

Home automation has evolved from simple electrical switching to advanced IoT-based systems capable of remote control and monitoring. With increasing demand for convenience, safety, and energy efficiency, modern smart homes are expected to support intuitive appliance operation, real-time energy awareness, and automatic emergency response. Traditional systems, however, still rely heavily on manual switches and lack integrated safety mechanisms, making them less effective in addressing everyday household challenges.

One of the key motivations behind this project is the need for touchless and hygienic control of appliances, especially in environments like kitchens, hospitals, and for elderly or differently-abled individuals who may face difficulty reaching physical switches. Another major concern is rising electricity consumption, which requires users to understand their usage patterns to prevent energy wastage. Furthermore, LPG or smoke leakage incidents continue to pose serious risks in households,

highlighting the need for proactive safety solutions rather than just reactive alerts.

This project aims to bridge these needs by developing a system that combines gesture-based appliance control, real-time energy monitoring using the ACS712 current sensor, and automatic gas response using the MQ135 sensor. The system not only detects gas leakage and triggers a buzzer warning but also automatically switches on an exhaust fan to reduce gas concentration. Additionally, all live data—including gas level, current, power, and total energy consumed—is uploaded to the Blynk IoT application, allowing users to monitor their home remotely with real-time analytics and graph visualization.

The motivation behind this work is to create a complete smart-home framework that improves comfort, enhances safety, and promotes energy-conscious living through advanced sensor integration and IoT connectivity.

B. Literature Review

Smart home automation has gained significant attention with the growth of IoT, embedded systems, and low-cost sensors. Several works propose gesture-based control of domestic loads to improve accessibility and hygiene, replacing conventional switches with hand movements captured by sensors or cameras [1]–[4]. Systems demonstrate that gesture interfaces are practical for controlling lights and fans, especially for elderly or differently-abled users, but they generally focus only on switching and do not incorporate energy analytics or gas-safety features.

In parallel, extensive research has explored energy monitoring using Hall-effect current sensors such as the ACS712, which provides galvanically isolated, linear analog output proportional to load current [5]–[7]. DIY and academic projects show ACS712 can be integrated with microcontrollers to measure AC/DC current, estimate instantaneous power, and compute total energy consumption for household loads [8]. However, most of these works present stand-alone meters or power-strip monitors rather than integrating current sensing with advanced control or safety mechanisms in a unified smart-home platform.

Gas and air-quality monitoring is another important research direction. The MQ135 gas sensor is widely used for detecting a range of harmful gases and monitoring air quality at low cost [9], [10]. Numerous LPG and smoke leakage detector designs employ MQ-series sensors with microcontrollers to trigger buzzers or LEDs when concentration exceeds a threshold, aiming to prevent domestic accidents [11]–[13]. These systems emphasize early

warning but usually stop at audible/visual alerts and do not automatically activate exhaust fans or ventilation systems to mitigate the hazard.

With the emergence of IoT platforms, Blynk-based home automation has become popular for remote control and monitoring of appliances. Blynk provides cloud connectivity, mobile dashboards, and device management for ESP32/ESP8266-based projects [14]. Several works demonstrate controlling relays and monitoring environmental parameters via Blynk on ESP32, enabling smartphone-based operation of lights, fans, and other loads [15]–[17]. Recent research further explores ESP32-Blynk systems for more advanced smart-home scenarios, highlighting reliability of Wi-Fi connectivity and the usefulness of real-time mobile dashboards [18]. Still, most projects limit themselves either to remote switching or simple sensor visualization and rarely integrate gesture interfaces, detailed energy sensing, and gas-safety actions together.

From this review, it is evident that existing literature treats gesture-based control, energy monitoring with ACS712, gas detection using MQ135, and Blynk-based IoT dashboards largely as separate problem domains. There is limited work that fuses these aspects into a single, low-cost smart-home system that (i) controls appliances via hand gestures, (ii) measures current, power, and total energy consumed, (iii) detects LPG/smoke levels with MQ135 and triggers both buzzer and automatic exhaust fan, and (iv) publishes real-time values and historical graphs (energy vs time, current vs time, power vs time, gas level vs time) to a Blynk mobile application. The present project addresses this gap by designing an integrated architecture that combines intuitive control, energy awareness, and active safety response within one ESP32-based platform.

C. Research Objectives

The primary aim of this research is to design and implement an integrated smart-home system that enhances convenience, safety, and energy efficiency. The specific objectives of the study are:

1. To develop a gesture-based appliance control mechanism that enables touch-free operation of household devices, providing a hygienic and user-friendly interface.
2. To monitor electrical parameters in real time, including current, power, and total energy consumption, using the ACS712 sensor for promoting energy-conscious behaviour.
3. To detect LPG/smoke concentration using the MQ135 sensor and trigger a buzzer alert when the gas level exceeds a predefined safety threshold.
4. To automatically activate an exhaust fan during high gas concentration, providing an immediate preventive response instead of relying only on alarm-based notifications.
5. To transmit all live sensor readings and appliance status to the Blynk IoT application via Wi-Fi, enabling remote visual monitoring and real-time graph analytics.
6. To integrate automation, energy monitoring, and safety in a single smart-home system, thereby improving overall functionality compared to existing standalone solutions

II. METHODOLOGY AND MODELING

The system integrates:

1. Gesture Control Module

Recognizes gestures to turn appliances ON/OFF.
Microcontroller (ESP32/Arduino) interprets gesture signals.

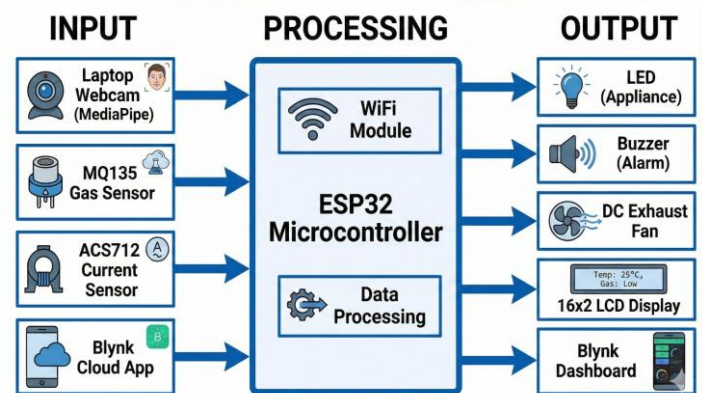
2. Energy Monitoring Module

Uses sensor ACS712
Measures current flow through appliances.
Calculates instantaneous power & total energy (Wh).
Displays values on serial monitor/LCD/Blynk dashboard.

3. LPG Leakage Detection Module

MQ-135 sensor continuously checks gas concentration.
If gas level crosses threshold, buzzer is activated along with exhaust fan.

A. Block Diagram of the proposed model system

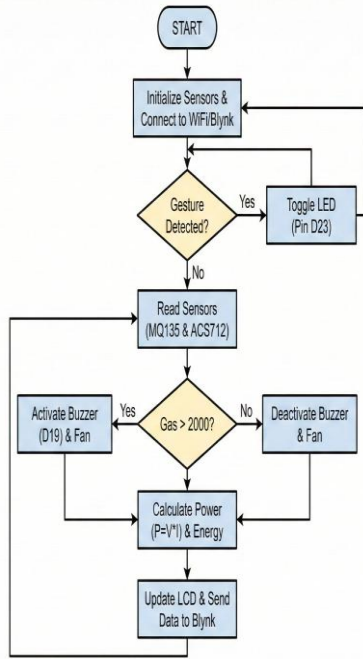


(fig.1.0)

WORKING MODEL:

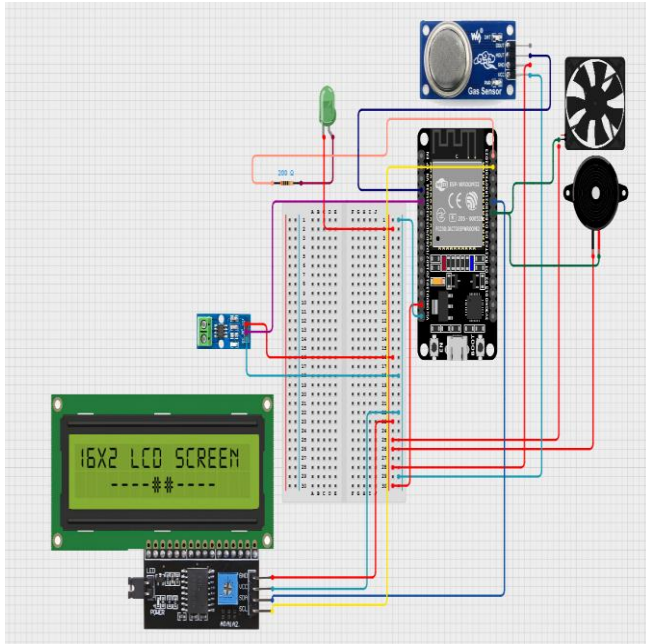
- The system operates by continuously monitoring hand gestures and sensor inputs through the ESP32 microcontroller. When a hand gesture is detected, the microcontroller interprets the pattern of movement and determines the intended control action. Based on the recognized gesture, the system switches the connected appliance (LED) either ON or OFF, providing a touch-free method of control.
- Simultaneously, the ACS712 current sensor measures the current flowing through the load in real time. Using the measured current and predefined voltage values, the system calculates instantaneous power as well as the total energy consumed over time. These calculated parameters help the user understand power usage and reduce unnecessary consumption.
- To ensure safety, the MQ135 gas sensor continuously monitors the surrounding air for the presence of LPG or smoke. When the gas concentration exceeds the preset threshold value, the system immediately triggers a buzzer to alert the user and automatically switches ON the exhaust fan in order to reduce gas concentration and prevent hazardous conditions.
- All system parameters including gas level, current, power, total energy, and appliance status are displayed live on the LCD and simultaneously updated on the Blynk IoT application via Wi-Fi. The Blynk dashboard presents both real-time readings and graphical visualizations, enabling users to monitor their home environment remotely with ease.

B. Flow Chart



(fig. 2.0)

C. Circuit Diagram of proposed model



(fig.3.0)

A) MQ135 Gas Sensor → ESP32

1. MQ135 VCC → +5 V supply rail
2. MQ135 GND → Common GND rail
3. MQ135 AO (Analog Output) → D34 (GPIO34)

B) ACS712 Current Sensor → ESP32

1. ACS712 VCC → +5 V supply rail
2. ACS712 OUT → D35 (GPIO35) of ESP32
3. ACS712 GND → Common GND rail

C) 16×2 I²C LCD → ESP32

1. LCD VCC → +5 V supply rail
2. LCD GND → Common GND rail

3. LCD SDA → GPIO21 (SDA) of ESP32
4. LCD SCL → GPIO22 (SCL) of ESP32

D) LED Load with Series Resistor → ESP32

1. D23 (GPIO23) of ESP32 → One end of current-limiting resistor (e.g., 330 Ω)
2. Other end of resistor → Anode (+) of LED / load
3. Cathode (-) of LED / load → Common GND rail

E) Buzzer → ESP32

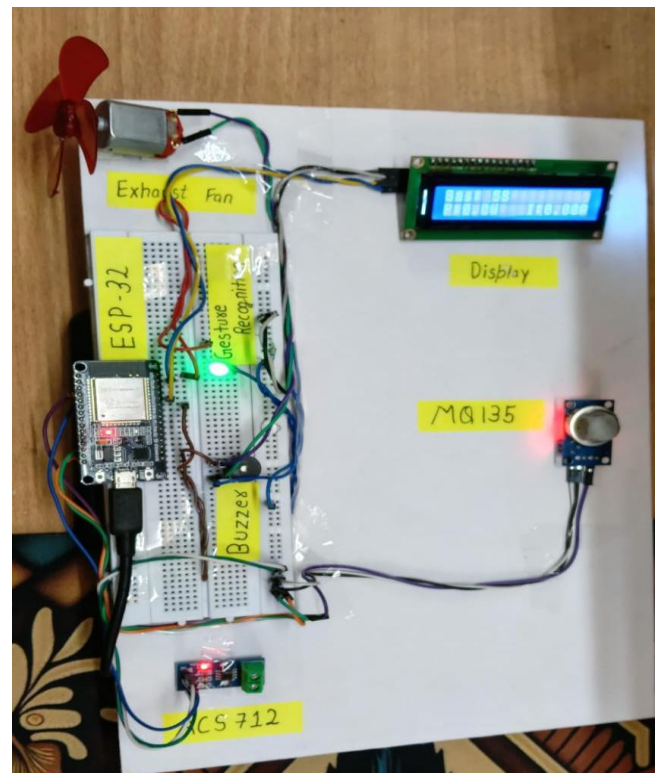
1. Buzzer positive (+) → D19 (GPIO19) of ESP32
2. Buzzer negative (-) → Common GND rail

F) Exhaust Fan (DC Motor) → ESP32

- Exhaust fan is connected in parallel with the buzzer:

 1. Fan positive (+) → D19 (GPIO19) of ESP32
 2. Fan negative (-) → Common GND rail

D. Layout of the proposed model



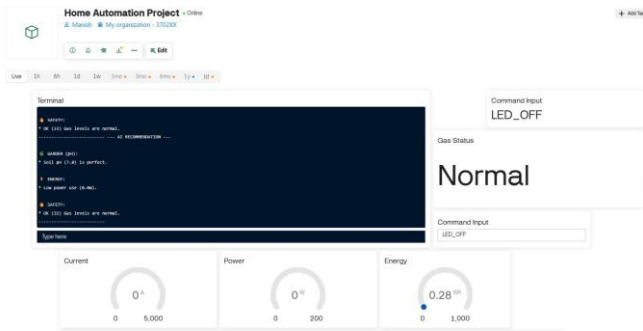
(fig. 4.0)

III. RESULTS

Experimental Results

Time	Current	Power	Cumulative Enegy	Gas Level(>2000 is unsafe)
t1	0.68A	72W	0.10Whr	604
t2	0.74A	79W	0.23Whr	723
t3	0.62A	63W	0.37Whr	440
Range	(0.60-0.75)A	(65-80)W	---	---

(table 1.0)



(fig.5.0 Blynk Dashboard)



(fig.5.1 Gas level v/s Time graph)



(fig.5.2 Instantaneous Current v/s Time graph)



(fig.5.3 Power v/s Time graph)



(fig.5.4 Total Energy graph v/s Time graph)

Comparison table

Features	Existing Home Automation Systems	Proposed Project (Your System)
Appliance Control	Mostly via manual switches / remote / mobile app	Hand-gesture-based contactless control
Energy Monitoring	Rarely included / external power meter needed	Built-in real-time monitoring of current, power & total energy
Gas / Smoke Detection	Some systems have gas alarm only	Gas detection + automatic exhaust fan + buzzer alert
IoT Connectivity	Limited / only device switching	Blynk IoT app with live values and graphs
Graphical Analytics	Generally not available	Graphs of energy, current, power, and gas level over time
Safety Response	Alert only (no automatic action)	Automatic exhaust during leakage

LCD Display	Optional in few systems	Built-in LCD for onsite live display
User Accessibility	Requires remote / app / switch	Fully touch-free operation for elderly & physically challenged people
Cost	High for multi-module systems	Low-cost integrated solution with multiple features
System Integration	Features available separately, not combined	Automation + safety + energy monitoring in a single system

(table 2.0)

IV. FUTURE POSSIBLE UPDATES:

1. Addition of multiple loads for gesture control.
2. Turning off the gas knob when the LPG leak detected.
3. Automatic turning off the knob of LPG when outside door is locked.
4. Auto cut of power in case of short circuit.

REFERENCES

[1] "Gesture-Based Home Automation Using Arduino," *Electronics For You*, Jul. 2025.

[2] "Home Automation Using Hand Gestures," *IRE Journals*, vol. X, no. Y, pp. 1–5.

[3] "Gesture Based Home Automation," *JournalSTD*, Apr. 2024.

[4] "Research Paper on Home Automation Using Hand Gesture," *IJIRT*, 2024.

[5] "ACS712 Current Sensor: Features, How It Works, Arduino Guide," Seeed Studio, 2020.

[6] "ACS712 Current Sensor Module for Arduino – Bidirectional Hall Effect Current Detection," Electroboat, product note.

[7] "ACS712 30A Ampere Current Sensor Breakout," Sumeet Instruments, product brief.

[8] "DIY Projects Using the ACS712 Current Sensor," HQ Online, blog article.

[9] "MQ135 Air Quality Sensor Datasheet: Working and Applications," Elprocus, 2021

[10] "MQ135 Air Quality/Gas Detector Sensor Module for Arduino," Robu.in, product description.

[11] "Gas Leakage Detector Using Arduino – Complete DIY Guide," CircuitDigest, Jun. 2025.

[12] "Evaluating MQ Sensors in Gas Leakage Detection Systems," *Journal of Software & Advanced Engineering Research*, vol. 11, no. 9, 2024

[13] "Sensor-Based Gas Leakage Detector System," Sathyabama Institute of Science and Technology, project report, 2021.

[14] "Blynk: Low-Code IoT Platform," Blynk Inc., official website.

[15] "IoT Home Automation Using Blynk & NodeMCU ESP8266," How2Electronics, project tutorial.

[16] "Smart Home Automation Using Blynk & ESP32 IoT Project," *Instructables*, 2020.

[17] "Blynk IoT Enabled Smart Home Automation System," *Journal of Computer Science & Communication Security*, Jun. 2024.

[18] D. Kakani et al., "IoT-Based Smart Home Automation Using ESP32 and Blynk," SSRN preprint, 2024.

CODE:

```
// =====  
// COMBINED CODE: Gesture + MQ135 + ACS72 + LCD  
// (CORRECTED FOR BLYNK AI + LCD + ALL FIXES)  
// =====  
  
// --- BLYNK CONFIGURATION ---  
#define BLYNK_TEMPLATE_ID "TMPL3PpUyWLwl"  
#define BLYNK_TEMPLATE_NAME "Home Automation Project"  
#define BLYNK_AUTH_TOKEN "-OEwcoIVmfUPMEJ1O05Mbsvd28ONR4aj"  
#define BLYNK_PRINT Serial  
  
// --- LIBRARIES ---  
#include <WiFi.h>  
#include <BlynkSimpleEsp32.h>  
#include <LiquidCrystal_I2C.h> // LCD Library  
  
// --- WIFI CREDENTIALS ---  
char wifi_ssid[] = "ARJUN";  
char wifi_pass[] = "arjun123";  
  
// --- HARDWARE PINS ---  
const int LED_PIN = 23; // Gesture LED  
const int MQ135_PIN = 34; // Gas Sensor (Analog)  
const int ACS712_PIN = 35; // Current Sensor (Analog)  
const int BUZZER_PIN = 19; // Buzzer  
  
// --- OBJECTS & VARIABLES ---  
BlynkTimer timer;  
LiquidCrystal_I2C lcd(0x27, 16, 2); // LCD Object (Address 0x27)  
  
// Sensor Settings  
int gasThreshold = 2000;  
float sensitivity = 0.185;  
float vRef = 3.3; // ESP32 is a 3.3V device  
  
// AI Energy Calculation  
float homeVoltage = 230.0; // Voltage for AI power calculation  
float totalEnergy = 0;  
unsigned long lastTime = 0;  
float myZeroPoint = 2.3260; // 3.3V / 2. If using 5V ACS712, change this to 2.5  
  
// =====  
// SETUP  
// =====  
void setup() {  
  Serial.begin(115200);  
  
  // -- Initialize Pins --  
  pinMode(LED_PIN, OUTPUT);  
  pinMode(BUZZER_PIN, OUTPUT);  
  pinMode(MQ135_PIN, INPUT);  
  pinMode(ACS712_PIN, INPUT);  
  
  digitalWrite(LED_PIN, LOW);  
  digitalWrite(BUZZER_PIN, LOW);  
}
```

```

// -- Initialize LCD --
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("System Booting..");
Serial.println("System Booting...");

// -- Connect to Blynk --
Blynk.begin(BLYNK_AUTH_TOKEN, wifi_ssid, wifi_pass);

lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Blynk Connected!");
Serial.println("Blynk Connected!");
delay(1000);

// -- Start Timer (Run sensors every 1000ms) --
timer.setInterval(1000L, runSensors);
lastTime = millis();
}

// =====
// MAIN LOOP
// =====
void loop() {
  Blynk.run();
  timer.run();
}

// =====
// TASK 1: BLYNK GESTURE
// =====
BLYNK_WRITE(V9) {
  String command = param.asString();
  if (command == "LED_ON") {
    digitalWrite(LED_PIN, HIGH);
  } else if (command == "LED_OFF") {
    digitalWrite(LED_PIN, LOW);
  }
}

// =====
// TASK 2: SENSORS & DISPLAY
// =====
void runSensors() {
  // --- A. READ SENSORS ---
  int gasValue = analogRead(MQ135_PIN);
  int adcValue = analogRead(ACS712_PIN);

  // --- B. CALCULATE CURRENT (Amps) ---
  float voltage = (adcValue / 4095.0) * vRef;

  // *** CHANGED THIS ***: Use myZeroPoint. Calibrate this value as needed.
  float current = (voltage - myZeroPoint) / sensitivity;

  // Noise Filter: Force to 0 if value is tiny
  if (abs(current) < 0.25) {
    current = 0.00;
  }
}

```

```

}

// --- C. CALCULATE POWER (W) & ENERGY (Wh) ---
// *** CHANGED THIS ***: Using homeVoltage for AI
float power = homeVoltage * abs(current);

// *** ADDED THIS BACK ***: Energy calculation for AI
unsigned long currentTime = millis();
float timeStepInHours = (currentTime - lastTime) / 3600000.0;
totalEnergy += power * timeStepInHours;
lastTime = currentTime;

// --- D. UPDATE BUZZER & BLYNK ALERTS ---
if (gasValue > gasThreshold) {
  digitalWrite(BUZZER_PIN, HIGH);
  Blynk.virtualWrite(V1, "ALERT");
} else {
  digitalWrite(BUZZER_PIN, LOW);
  Blynk.virtualWrite(V1, "Normal");
}

// --- E. SEND ALL DATA TO BLYNK (FOR AI) ---
// *** THIS BLOCK IS NOW INSIDE THE FUNCTION ***
Blynk.virtualWrite(V5, gasValue); // Send raw gas to AI
Blynk.virtualWrite(V6, power); // Send power to AI
Blynk.virtualWrite(V7, totalEnergy); // Send energy to AI
Blynk.virtualWrite(V10, abs(current)); // Send current for dashboard

// --- F. UPDATE LCD ---
// *** THIS BLOCK IS NOW INSIDE THE FUNCTION ***
lcd.clear();
// Row 0: Gas Status
lcd.setCursor(0, 0);
lcd.print("Gas: ");
lcd.print(gasValue);
if(gasValue > gasThreshold) lcd.print(" !"); // Exclamation if danger

// Row 1: Power Info
lcd.setCursor(0, 1);
lcd.print("P:");
lcd.print(power, 1); // Show 1 decimal place
lcd.print("W");

lcd.setCursor(9, 1); // Move to right side
lcd.print("I:");
lcd.print(abs(current), 2); // Show 2 decimal places
lcd.print("A");

// --- G. PRINT TO SERIAL MONITOR ---
// *** THIS LINE IS NOW INSIDE THE FUNCTION ***
Serial.printf("Gas: %d | Power: %.1f W | Current: %.2f A | Energy: %.4f Wh\n",
  gasValue, power, abs(current), totalEnergy);

} // <-- *** THIS IS THE CORRECT, FINAL BRACE for runSensors ***

```